A Communication-Realistic Sim-to-Sim Validation Framework for Reinforcement Learning Humanoid Locomotion Using Digital Twins

Juan Andres Mendez[1*] and Carlos Lozano-Garzon[1†]

[1*]Systems and Computing Engineering Department, Universidad de los Andes, Street, Bogota, 100190, Colombia.

*Corresponding author(s). E-mail(s): ja.mendez@uniandes.edu.co;
Contributing authors: calozanog@uniandes.edu.co;
†These authors contributed equally to this work.

Abstract

Humanoid robot locomotion remains a challenging problem for reinforcement learning due to the complexity of high-degree-of-freedom articulated systems, safety risks during experimental training, and the persistent simulation-to-reality transfer gap. These challenges are particularly relevant in research and educational contexts where access to physical humanoid platforms is limited. This paper presents an integrated framework for developing and validating humanoid locomotion policies that combines digital twins, reinforcement learning training, and communication-realistic validation prior to hardware deployment. The proposed pipeline trains goal-directed locomotion policies for the Unitree G1 humanoid robot in a high-fidelity simulation environment using Proximal Policy Optimization operating on proprioceptive observations across 29 actuated degrees of freedom. To reduce deployment risks, the framework incorporates a simulation-to-simulation validation stage that evaluates trained policies using authentic ROS 2 middleware communication, enabling testing under realistic control conditions before execution on the physical robot. Experimental results demonstrate stable bipedal locomotion and achieve an 87% goal success rate in point-to-point navigation tasks. Additional exploratory experiments on obstacle-aware navigation reveal important challenges in reward design when combining goal-seeking and collision-avoidance objectives. The proposed framework provides a practical methodology for safer and more reproducible reinforcement learning experimentation in humanoid robotics.

Keywords: Humanoid Robots; Deep Reinforcement Learning; Robot Locomotion; simulation-to-reality transfer; Digital Twins; ROS 2 Middleware; Social Navigation

1 Introduction

Humanoid robots are among the most complex and promising domains in modern robotics. Their ability to operate in environments designed for humans makes them particularly attractive for applications ranging from service robotics to human–robot interaction and collaborative automation. However, achieving robust locomotion on humanoid platforms remains challenging due to the large number of degrees of freedom, nonlinear dynamics, and the need to maintain dynamic balance while interacting with complex environments [1], [2].

Recent advances in physics-based simulation and deep reinforcement learning (RL) have significantly expanded the possibilities for developing locomotion control policies in simulated environments prior to physical deployment. High-fidelity simulation platforms enable the creation of digital twins that replicate the kinematic structure, dynamics, and sensing capabilities of real robotic systems with greater realism [3]. In parallel, reinforcement learning algorithms have demonstrated promising results in synthesizing locomotion behaviors for robots with complex articulated structures, allowing control policies to emerge through interaction with simulated environments rather than explicit programming [4].

Despite these advances, transferring learned policies from simulation to real robotic systems remains a significant challenge. This problem, commonly referred to as the simulation-to-reality (sim-to-real) gap, arises from discrepancies between simulated and physical environments, including modeling inaccuracies, sensor noise, actuator limitations, and environmental variability. Even small differences between simulation and reality can cause learned policies to degrade or fail when deployed on physical hardware.

An additional challenge arises from the fact that most reinforcement learning locomotion studies evaluate trained policies directly within simulation environments that bypass the software architectures used in real robotic systems. In practice, humanoid robots rely on middleware frameworks that manage communication between sensors, control modules, and actuators. These middleware layers introduce communication delays, message serialization, and asynchronous processing effects that are typically not represented during reinforcement learning training or evaluation. As a result, policies that perform well in native simulation environments may exhibit unexpected behavior when integrated with the software stacks used for real robotic deployment.

Addressing this validation gap is particularly important in research and educational environments where access to humanoid hardware platforms is limited and experimental failures can result in costly damage or extended system downtime. Under these constraints, development pipelines that enable extensive experimentation and validation prior to hardware deployment are essential for accelerating research while preserving the integrity of robotic platforms.

This paper proposes an integrated framework for training and validating humanoid locomotion policies that combines digital twins, deep reinforcement learning, and communication-realistic validation. The framework is implemented using a high-fidelity digital twin of the Unitree G1 humanoid robot and trains goal-directed locomotion policies using Proximal Policy Optimization operating directly on joint-level control actions. To reduce deployment risks, the proposed pipeline incorporates a simulation-to-simulation validation stage in which trained policies interact with the robot model through authentic ROS 2 middleware communication, reproducing the control architecture used in real robotic systems.

The proposed approach is evaluated through a set of locomotion experiments in which the robot learns goal-directed point-to-point navigation using proprioceptive observations across 29 actuated degrees of freedom. The learned policy achieves stable bipedal locomotion with an 87% success rate in reaching navigation targets. In addition, exploratory experiments investigate the integration of LiDAR-based perception for obstacle-aware navigation. These experiments reveal important challenges in reward design when combining goal-seeking and collision-avoidance objectives, highlighting the complexity of multi-objective learning for humanoid navigation.

The contributions of this work are threefold:

1. **A communication-realistic simulation-to-simulation validation framework** that evaluates reinforcement learning locomotion policies using authentic ROS 2 middleware communication prior to hardware deployment.
2. **An integrated development pipeline for humanoid locomotion research** that combines digital twins, reinforcement learning training, and middleware-based validation.
3. **Experimental validation of goal-directed locomotion policies** trained with Proximal Policy Optimization on a high-fidelity digital twin of the Unitree G1 humanoid robot.

By incorporating realistic communication architectures into the validation process, the proposed framework provides an intermediate evaluation stage between reinforcement learning training and physical deployment, enabling safer and more reliable development of locomotion policies for humanoid robotic systems.

2 Related Work

Research on humanoid locomotion has evolved across several complementary directions, including reinforcement learning–based control methods, high-fidelity simulation environments, and techniques for transferring learned policies from simulation to real robotic platforms. In addition, modern robotic systems increasingly rely on middleware architectures that coordinate sensing, control, and actuation processes across distributed software components. Although these research areas have advanced significantly in recent years, their integration remains limited in many locomotion development pipelines. This section reviews the literature most closely related to the present work in four areas: reinforcement learning for humanoid locomotion, digital-twin-based robotics experimentation, simulation-to-reality transfer methods, and middleware-based robotic architectures.

2.1 Reinforcement Learning for Humanoid Locomotion

Humanoid locomotion remains one of the most challenging control problems in robotics due to the large number of degrees of freedom, nonlinear dynamics, and the requirement to maintain dynamic balance during motion. Traditional approaches to humanoid locomotion control have relied on hierarchical architectures that decompose the problem into multiple layers, including high-level path planning, trajectory generation, and low-level joint control [1], [2]. While these methods have achieved considerable success, they often require extensive manual engineering and careful tuning of intermediate control modules.

Recent advances in deep reinforcement learning (RL) have introduced alternative approaches in which locomotion policies are learned through interaction with simulated environments. In contrast to traditional model-based control strategies, RL methods enable control policies to emerge from data-driven learning rather than from explicit programming of gait patterns. Among these methods, policy gradient algorithms such as Proximal Policy Optimization (PPO) have become widely used due to their stability and effectiveness in continuous control tasks [4], [5].

Large-scale simulation environments have enabled the successful application of reinforcement learning to a variety of legged robots, including quadrupeds and humanoids. These approaches have demonstrated that complex locomotion behaviors can be learned through iterative training processes involving millions of simulation steps [6], [7]. However, many existing approaches rely on hierarchical learning strategies, imitation learning, or carefully designed curriculum learning procedures to stabilize training and guide the emergence of locomotion behaviors [8], [9]. End-to-end approaches that directly learn joint-level control policies for humanoid robots remain comparatively less explored due to the high dimensionality and instability of the control problem.

2.2 Digital Twins for Robotics

The concept of digital twins has emerged as a powerful paradigm for representing physical systems within simulated environments. A digital twin is a virtual replica of a physical system that reproduces its geometric structure, dynamic properties, sensing capabilities, and interactions with the surrounding environment [10], [11]. In robotics research, digital twins enable the development and testing of algorithms without exposing physical hardware to the risks associated with early-stage experimentation.

Recent advances in robotics simulation platforms have significantly improved the realism of simulated environments by incorporating high-fidelity physics engines that model rigid-body dynamics, friction, contact interactions, and sensor behavior [3], [12]. These capabilities allow digital twins to approximate the behavior of real robotic systems with increasing accuracy.

For humanoid robots, digital twins are particularly valuable due to the complexity and fragility of these platforms. Experimental failures during locomotion training can result in falls, actuator overload, or mechanical damage, leading to costly repairs and system downtime. By enabling large-scale experimentation in safe virtual

environments, digital twins enable reinforcement learning algorithms to perform extensive training iterations before deploying policies to physical robots. This approach significantly reduces development risks while accelerating experimentation cycles [13].

2.3 Simulation-to-Reality Transfer

Although simulation-based training offers substantial advantages, transferring learned policies from simulation to physical robots remains a major challenge commonly referred to as the simulation-to-reality (sim-to-real) gap. This gap arises from discrepancies between simulated and real environments, including modeling inaccuracies, unmodeled dynamics, sensor noise, and actuator limitations [14].

Several techniques have been proposed to address this challenge. Domain randomization introduces variability in simulation parameters during training, thereby making learned policies robust to environmental uncertainty [15]. System identification methods attempt to reduce discrepancies between simulated and real systems by calibrating simulation parameters using experimental measurements. Other approaches combine reinforcement learning with model-based control methods to improve stability and generalization during deployment [16].

Despite these advances, most sim-to-real research assumes a direct transition from simulation training to physical deployment once a policy achieves satisfactory performance in simulation. In practice, however, robotic systems operate within complex software architectures that include middleware communication layers, distributed processing nodes, and real-time control loops. These architectural elements can introduce additional latency, synchronization issues, and communication constraints that are typically not represented during reinforcement learning training.

2.4 Robotics Middleware and Validation Pipelines

Modern robotic systems rely heavily on middleware frameworks that coordinate communication between sensing modules, control algorithms, and hardware interfaces. Among these frameworks, ROS 2 has become widely adopted in robotics research and development due to its support for distributed systems, real-time communication, and modular software architectures [17].

In practical robotic deployments, control policies interact with sensors and actuators through middleware layers that manage message passing, synchronization, and system integration. However, reinforcement learning pipelines frequently bypass these middleware layers by interacting directly with simulator APIs. As a result, the communication characteristics present in real robotic systems—such as message serialization, asynchronous processing, and network latency—are typically absent during policy training and evaluation.

This discrepancy introduces a methodological gap between simulation-based training and real robotic deployment. Policies validated exclusively in simulation environments may exhibit unexpected behavior when integrated into real robotic software stacks, where timing and communication constraints affect control stability and responsiveness [18]. Addressing this gap is particularly important for humanoid robots, where control instability may lead to falls or hardware damage.

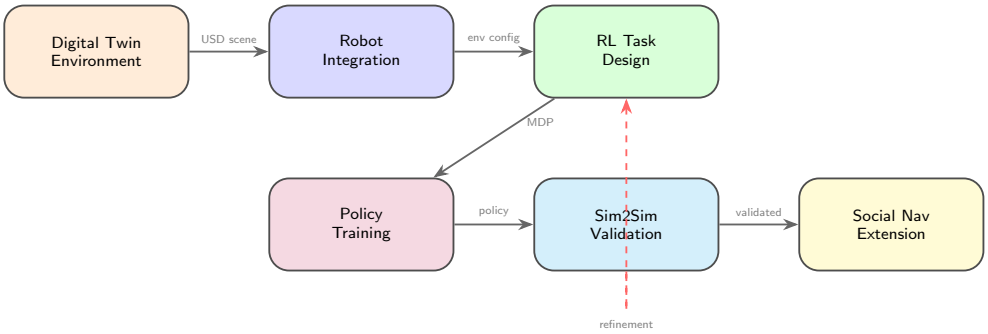


Fig. 1: Framework overview: (1) digital twin construction, (2) RL training with goal-directed commands, (3) sim-to-sim validation via ROS 2 middleware. Arrows indicate information flow; double arrows indicate the policy inference loop.

Although several studies have investigated simulation-based training for locomotion policies, comparatively little work has focused on incorporating realistic middleware communication into reinforcement learning validation pipelines. Integrating these communication architectures into simulation-based experimentation could provide a more reliable intermediate step between policy training and physical deployment.

In this context, the present work introduces a communication-realistic simulation-to-simulation validation framework in which reinforcement learning policies interact with a digital twin through authentic ROS 2 middleware communication. By reproducing the software architecture of real robotic systems, this approach enables evaluation of locomotion policies under communication conditions that more closely resemble those encountered in real deployment. This intermediate validation stage aims to reduce deployment risks and improve the reliability of reinforcement learning–based locomotion control for humanoid robots.

3 Framework Architecture and Methodology

This section describes the proposed framework for training and validating locomotion policies for the Unitree G1 humanoid robot using reinforcement learning. The framework integrates high-fidelity simulation, digital twin modeling, reinforcement learning training pipelines, and a simulation-to-simulation validation stage that leverages authentic robotic middleware communication.

The objective of the framework is twofold:

1. Enable safe and scalable training of locomotion policies for high-degree-of-freedom humanoid robots.
2. Provide an intermediate validation stage that reduces risks prior to deployment on physical hardware.

Figure 1 illustrates the overall framework structure. The system is composed of four main components:

- A digital twin simulation environment implemented in NVIDIA Isaac Lab.
- A reinforcement learning training pipeline based on Proximal Policy Optimization.
- A goal-directed locomotion task definition
- A simulation-to-simulation validation layer based on ROS 2 Humble communication

3.1 Digital Twin of the Unitree G1

The training environment is built using NVIDIA Isaac Lab, which provides GPU-accelerated physics simulation and reinforcement learning infrastructure for robotic systems.

A detailed digital twin of the Unitree G1 humanoid robot was implemented to reproduce the mechanical and control characteristics of the physical platform. The model includes:

- Full articulated kinematic structure
- 29 actuated joints
- Joint limits and actuator constraints
- Contact dynamics for bipedal locomotion
- Proprioceptive sensing

The digital twin serves as the training environment for the reinforcement learning agent, enabling the collection of large amounts of interaction data while avoiding the safety risks and hardware degradation associated with real-world experimentation.

The simulation environment models terrain interactions and gravitational dynamics, enabling the agent to learn stable locomotion behaviors through repeated episodes.

3.2 Reinforcement Learning Formulation

The locomotion task is formulated as a continuous control reinforcement learning problem.

At each timestep t, the agent observes a state vector s_t composed of proprioceptive information from the robot and produces an action vector a_t representing desired joint commands.

3.2.1 Observation Space

The state vector $\mathbf{s}_t \in \mathcal{S} \subseteq \mathbb{R}^{125}$ concatenates all proprioceptive signals available on physical hardware:

$$\mathbf{s}_t = \left[\mathbf{p}_{goal}^{base},\ \mathbf{v}_{base},\ \boldsymbol{\omega}_{base},\ \mathbf{g}_{proj},\ \mathbf{q},\ \dot{\mathbf{q}},\ \mathbf{v}_{cmd},\ \mathbf{a}_{t-1} \right]^{\top} \tag{1}$$

Table 1 details each component and its dimensionality. These observations provide sufficient information for the policy to infer the robot's current locomotion state and the target's direction.

3.2.2 Action Space

The action vector comprises joint-level control commands for the robot's actuated degrees of freedom.

Table 1: State space components ($n_s = 125$)

Component	Dim.	Description
$\mathbf{p}_{goal}^{base}$	2	Goal position in robot base frame (x, y) m
$\mathbf{v}_{base}$	3	Base linear velocity ($\mathrm{m\,s^{-1}}$)
$\boldsymbol{\omega}_{base}$	3	Base angular velocity ($\mathrm{rad\,s^{-1}}$)
$\mathbf{g}_{proj}$	3	Projected gravity vector (orientation encoding)
$\mathbf{q}$	29	Joint positions relative to default (rad)
$\dot{\mathbf{q}}$	29	Joint velocities ($\mathrm{rad\,s^{-1}}$)
$\mathbf{v}_{cmd}$	3	Velocity commands $(v_x^{cmd}, v_y^{cmd}, \omega_z^{cmd})$
$\mathbf{a}_{t-1}$	29	Previous joint commands
Total	**125**	All signals available on physical Unitree G1

For the Unitree G1, the policy outputs control signals for 29 actuated joints, enabling coordinated full-body locomotion behavior.

3.2.3 Policy Representation

The policy $\pi_\theta : \mathcal{S} \to \mathcal{A}$ is represented as a stochastic multilayer perceptron parameterized by weights θ:

$$\pi_\theta(\mathbf{a} \mid \mathbf{s}) = \mathcal{N}\big(\boldsymbol{\mu}_\theta(\mathbf{s}),\, \mathrm{diag}(\boldsymbol{\sigma}^2)\big) \tag{2}$$

where $\boldsymbol{\mu}_\theta(\mathbf{s}) \in \mathbb{R}^{29}$ is the mean output by the actor network and $\boldsymbol{\sigma} \in \mathbb{R}^{29}$ are state-independent learned standard deviations. The actor network has three fully connected hidden layers of widths $[512, 256, 128]$ with ELU activations, mapping the 125-dimensional state to 29 joint position targets. During evaluation the deterministic mean $\boldsymbol{\mu}_\theta(\mathbf{s})$ is used.

3.2.4 Training Algorithm

The policy is optimized using Proximal Policy Optimization (PPO), a widely used policy-gradient algorithm for continuous control problems.

PPO maximizes a clipped surrogate objective that constrains policy updates, improving training stability and preventing destructive policy shifts during learning.

The training process involves repeated rollout collection in the simulated environment, followed by gradient updates using minibatch stochastic optimization.

3.3 Goal-Directed Locomotion Task

The primary task considered in this work is goal-directed point-to-point navigation.

At the beginning of each episode:

- The humanoid robot is placed at an initial position.
- A navigation goal is randomly generated within the environment.

The objective of the policy is to produce stable locomotion behavior that moves the robot toward the target location while maintaining balance.

Episodes terminate when:

- The robot reaches the target region
- The robot falls
- A maximum timestep limit is reached

A key design contribution is the goal-directed velocity command module, which transforms a goal position into locomotion velocity commands rather than issuing velocity commands directly. The transformation incorporates distance-based speed scaling and heading-aware modulation:

$$s_{dist} = v_{min} + (1 - v_{min}) \cdot \min\big(1,\, d_{goal}/d_{thr}\big) \tag{3}$$

$$s_{head} = 0.4 + 0.6\cos(\theta_{err}) \tag{4}$$

$$v_x^{cmd} = \max\big(0,\; s_{dist} \cdot s_{head} \cdot v_{max}\big) \tag{5}$$

$$\omega_z^{cmd} = \mathrm{clip}\big(K_\omega\, \theta_{err},\; \pm\omega_{max}\big) \tag{6}$$

where $v_{min} = 0.2$, $d_{thr} = 3.0\,\mathrm{m}$, $v_{max} = 1.0\,\mathrm{m\,s^{-1}}$, $K_\omega = 5.0$, $\omega_{max} = 1.0\,\mathrm{rad\,s^{-1}}$, and θ_{err} is the heading error to the goal. The constraint $v_x^{cmd} \geq 0$ forces the robot to rotate toward goals rather than walk away, eliminating a major source of early-training failures.

The task formulation avoids a hierarchical decomposition into separate gait-generation and trajectory-planning modules. Instead, locomotion and navigation behaviors are learned end-to-end through reinforcement learning.

3.4 Reward Design

Reward design plays a critical role in reinforcement learning for robotic locomotion, as it directly shapes the policy's learned behavior.

The reward function used for goal-directed locomotion combines several components that encourage forward motion, stability, and goal completion.

The main reward terms include:

- Goal distance reward, encouraging movement toward the target position
- Velocity alignment reward, promoting motion in the direction of the goal
- Posture stability reward, penalizing excessive tilt or unstable orientations
- Energy regularization, discouraging unnecessarily large joint commands
- Fall penalty, applied when the robot loses balance

The overall reward function can be expressed as:

$$R_t = \sum_{i=1}^{N} w_i \cdot r_i(\mathbf{s}_t, \mathbf{a}_t) \tag{7}$$

Table 2 provides the complete specification for the goal-directed locomotion task. The velocity-tracking group (combined weight 2.0) dominates the goal-reaching group (combined weight 0.5) at a 4:1 ratio, ensuring that locomotion stability is learned first while goal-seeking provides a gentle guidance signal.

Table 2: Reward specification for goal-directed locomotion. d_{goal} = distance to goal (m); θ_{err} = heading error (rad); v_x, ω_z = forward/yaw velocity; c_i = foot contact indicator.

Term	w_i	Definition
Locomotion objectives		
Lin. velocity tracking	$+1.0$	$\exp\!\big(-\lVert\mathbf{v}_{xy} - \mathbf{v}_{xy}^{cmd}\rVert^2/0.25\big)$
Ang. velocity tracking	$+1.0$	$\exp\!\big(-(\omega_z - \omega_z^{cmd})^2/0.25\big)$
Feet air time	$+0.75$	$\sum_i \max(0,\, t_{air,i} - 0.1)$
Goal-reaching objectives		
Goal distance (coarse)	$+0.3$	$\exp\!\big(-d_{goal}^2/25\big) \quad (\sigma = 5\,\mathrm{m})$
Goal distance (fine)	$+0.2$	$\exp\!\big(-d_{goal}^2/9\big) \quad (\sigma = 3\,\mathrm{m})$
Heading alignment	$+0.3$	$\exp\!\big(-\theta_{err}^2/0.25\big)$
Regularization penalties		
Action rate	-0.005	$-\lVert\mathbf{a}_t - \mathbf{a}_{t-1}\rVert^2$
Arm joint deviation	-0.1	$-\lVert\mathbf{q}_{arms} - \mathbf{q}_{def}\rVert^2$
Feet slide	-0.1	$-\sum_i \lVert\mathbf{v}_{foot,i}\rVert \cdot c_i$
Flat orientation	-1.0	$-\lVert\mathbf{g}_{proj,xy}\rVert^2$
Backwards velocity	-0.5	$-\mathbb{1}[v_x < -0.1]$
Termination	-200	$-\mathbb{1}[\text{fall detected}]$

where w_i are weighting coefficients that balance the contribution of each component.

3.4.1 Reward Design Challenges in Social Navigation

Exploratory experiments were conducted to extend the framework toward social navigation scenarios involving obstacle perception.

In this extension, LiDAR-based perception was integrated into the observation space, and additional reward terms were introduced to encourage obstacle avoidance.

These included:

- Proximity penalties for approaching obstacles
- Collision penalties
- Safe distance rewards

However, the introduction of multiple competing objectives—goal-seeking and obstacle-avoidance—created significant challenges in reward balancing. During preliminary experiments, the learned policy achieved only a 1% goal success rate, indicating that the reward gradients favored overly conservative behaviors that prevented effective navigation.

Analysis of the training dynamics revealed a gradient imbalance between goal-oriented rewards and proximity penalties, leading the agent to prioritize obstacle avoidance over goal completion.

This outcome highlights a well-known difficulty in reinforcement learning: multi-objective reward shaping can introduce conflicting optimization signals, requiring careful normalization and scaling of reward components.

While effective social navigation remains future work, the experiments conducted in this study provided valuable insight into the sensitivity of humanoid locomotion policies to reward design.

3.5 Simulation-to-Simulation Validation with ROS 2

To reduce risks before deploying policies on the physical robot, the framework incorporates a simulation-to-simulation (sim2sim) validation stage.

At this stage, the trained locomotion policy is evaluated in a separate simulation pipeline that uses authentic ROS 2 Humble communication interfaces, thereby replicating the messaging architecture of the physical robot.

The validation system includes:

- ROS 2 nodes for control commands
- ROS 2 topics for sensor data streaming
- Middleware-based message passing between simulation components

This design introduces realistic communication constraints, including serialization overhead and message-based control loops, which are typically absent in tightly coupled training simulations.

By evaluating policies under these conditions, the framework enables early identification of integration issues that could arise during real-world deployment.

The sim2sim stage therefore, acts as an intermediate validation layer, bridging the gap between reinforcement learning simulation environments and physical robotic systems

4 Experimental Setup

This section describes the experimental configuration used to train and evaluate the locomotion policies proposed in this work. The experimental setup includes the simulation environment, reinforcement learning training parameters, evaluation scenarios, and performance metrics used to assess the learned policies.

4.1 Simulation Environment

All experiments were conducted using NVIDIA Isaac Lab, a GPU-accelerated reinforcement learning framework built on top of NVIDIA Omniverse Isaac Sim. This platform provides a high-fidelity physics simulation that models articulated robots and contact-rich locomotion dynamics.

The simulation environment includes a digital twin of the Unitree G1 humanoid robot, configured to match the mechanical and kinematic specifications of the real platform.

Key properties of the simulated robot include:

- 29 actuated joints used for locomotion control
- Articulated rigid-body dynamics
- Contact-based foot interaction with the ground
- Joint limits and actuator constraints

The physics engine simulates gravity, rigid body dynamics, and contact interactions required for stable bipedal locomotion. The environment allows large-scale parallel simulation, enabling efficient reinforcement learning training.

Each training episode initializes the robot at a predefined starting pose with small randomized perturbations to improve robustness.

4.2 Reinforcement Learning Training Configuration

The locomotion policy was trained using Proximal Policy Optimization (PPO), a policy-gradient algorithm widely used for continuous robotic control tasks.

Training was performed using the reinforcement learning tools available within Isaac Lab.

The neural policy architecture consists of a multilayer perceptron that maps the observation vector to joint-level control commands.

4.2.1 Policy Network Architecture

The policy network includes:

- Input layer corresponding to the observation vector
- Two hidden fully connected layers
- Nonlinear activation functions
- Output layer producing continuous joint commands

Both the policy network and the value function are trained simultaneously as required by PPO.

4.2.2 Training Parameters

The main hyperparameters used during training are summarized in Table 3

Table 3: Training Parameters

Parameter	Value
Algorithm	PPO (RSL-RL)
Policy architecture	MLP, layers $[512, 256, 128]$
Activation function	ELU
Simulation timestep	0.02 s (50 Hz)
Episode length	1 000 steps (20 s)
Parallel environments	512
Discount factor γ	0.99
GAE lambda λ	0.95
PPO clip ε	0.2
Learning rate	1×10^{-3} (adaptive)
Mini-batch size	4 096
Update epochs / iteration	5
Total training iterations	1 500
GPU	NVIDIA RTX 4080 Super

Training was performed on a GPU to enable parallel rollout generation.

The agent interacts with the environment for multiple training iterations until locomotion behaviors emerge and converge.

Domain Randomization

To promote transfer robustness, each training episode applies randomization to physical parameters sampled at reset time (Table 4).

Table 4: Domain randomization parameters applied during training

Parameter	Distribution	Rationale
Link masses	$\mathcal{U}[0.85m,\ 1.15m]$	Manufacturing tolerances
Ground friction	$\mathcal{U}[0.5,\ 1.2]$	Surface material variation
Joint damping	$\mathcal{U}[0.8d,\ 1.2d]$	Actuator wear
Motor strength	$\mathcal{U}[0.9\tau_0,\ 1.1\tau_0]$	Battery/driver variation
Obs. noise	$\mathcal{N}(0,\ 0.01^2)$ rad	Encoder quantization
Action delay	$\mathcal{U}\{1, 2, 3\}$ steps	Communication latency

4.3 Navigation Task Configuration

The evaluation task focuses on goal-directed locomotion.

At the beginning of each episode:

1. The humanoid robot is initialized in an upright stance.
2. A target position is randomly generated within the simulated environment.

The robot must learn to move toward the goal while maintaining balance and stable locomotion.

The episode terminates under three possible conditions:

- The robot reaches the goal region.
- The robot falls or loses balance.
- The maximum timestep limit is reached.

The goal position varies across episodes in order to evaluate the policy's ability to generalize across different navigation directions.

4.4 Simulation-to-Simulation Validation Architecture

To evaluate the deployability of the learned policies, the framework incorporates a simulation-to-simulation validation stage.

This stage connects the locomotion policy to a robotics middleware environment implemented using ROS 2 Humble, the same communication framework used in the physical robot.

In this validation configuration:

- The reinforcement learning policy runs as a control node.
- Commands are transmitted through ROS 2 topics.
- Sensor observations are published through the same middleware interface.

This architecture introduces realistic communication constraints and message-based control loops that replicate the structure of a real robotic system.

The sim-to-sim validation environment, therefore, enables verification of the control pipeline prior to deployment on the physical platform.

4.5 Evaluation Metrics

To quantitatively evaluate locomotion performance, several metrics were defined.

4.5.1 Goal Success Rate

The goal success rate measures the percentage of episodes in which the robot successfully reaches the target location before the episode terminates.

This metric quantifies the policy's effectiveness in achieving goal-directed locomotion.

4.5.2 Locomotion Stability

Stability is evaluated by measuring the number of episodes in which the robot maintains an upright posture without falling.

Additional indicators, such as torso orientation and base angular velocity, are also monitored to identify unstable behaviors.

4.5.3 Energy Efficiency

Energy efficiency is estimated by the magnitude of the joint control commands generated by the policy.

Lower average control effort indicates more efficient locomotion strategies.

4.5.4 Episode Duration

The average episode duration provides insight into whether failures occur early (e.g., falls) or whether the robot remains operational for extended periods.

4.6 Experimental Scenarios

Two main experimental scenarios were considered in this study.

Scenario 1 – Goal-Directed Locomotion In the first scenario, the reinforcement learning policy is trained to reach randomly generated goal positions without obstacles.

This configuration evaluates the feasibility of end-to-end locomotion learning for a high-DOF humanoid robot.

Scenario 2 – LiDAR-Based Social Navigation (Exploratory) A second experimental configuration introduces LiDAR perception into the observation space in order to explore obstacle-aware navigation behaviors.

In this scenario:

- Obstacles are placed in the environment
- LiDAR observations are provided to the policy
- Additional reward terms penalize collisions and proximity

This exploratory experiment aims to evaluate whether end-to-end reinforcement learning can simultaneously solve locomotion and obstacle avoidance tasks.

5 Results and Discussion

This section presents the experimental results obtained from the training and evaluation of the locomotion policies developed in this work. The analysis focuses on three aspects: the performance of the learned locomotion policy, the validation of the communication-realistic sim-to-sim pipeline, and the exploratory evaluation of obstacle-aware navigation using LiDAR perception.

5.1 Locomotion Policy Performance

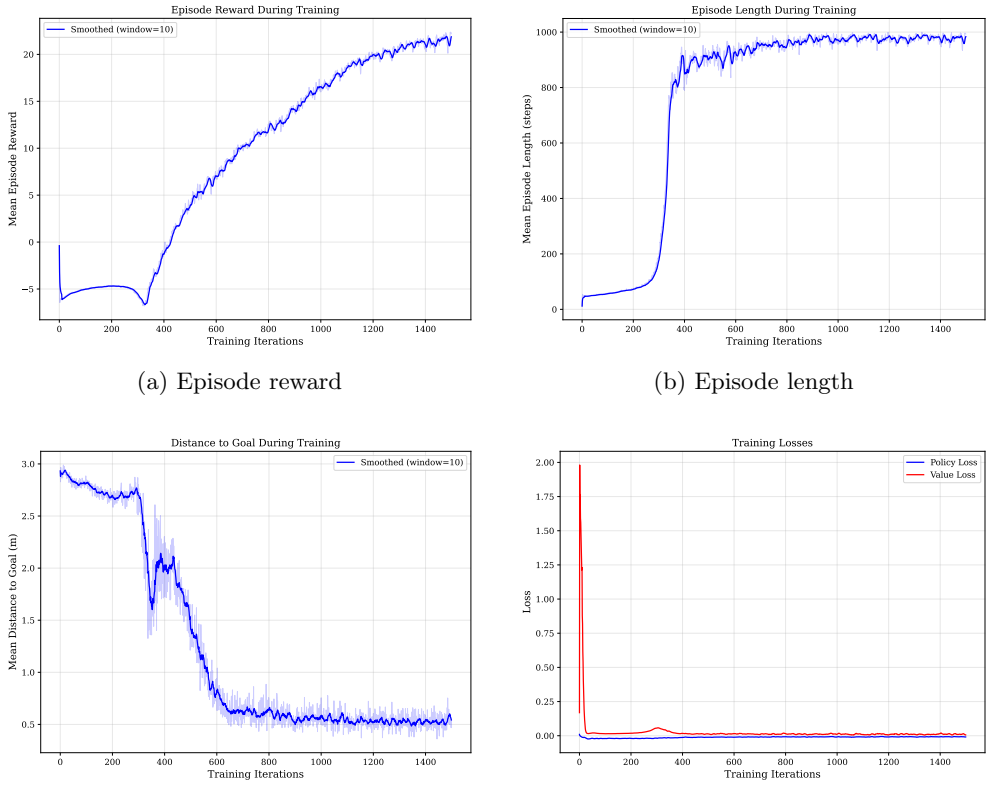
The reinforcement learning policy trained using Proximal Policy Optimization (PPO) successfully learned stable bipedal locomotion and goal-directed navigation behaviors in simulation.

Across the evaluation episodes, the policy achieved a goal success rate of 87%, indicating that the robot was able to consistently reach the target position while maintaining balance. Table 5 summarizes performance metrics measured over 100 evaluation episodes with fixed random seeds distinct from training.

Table 5: Evaluation metrics over 100 episodes (same simulator, different seeds from training)

Metric	Value	Note
Mean episode reward	20.48 ± 3.21	Stable positive locomotion
Mean episode length	921.5 ± 89.4 steps	92% of 1 000-step maximum
Timeout rate	87.0%	Episodes reaching time limit
Fall rate	8.2%	Early termination due to fall
Goal success rate	87.0%	Within 0.3 m of target
Mean distance to goal	0.42 ± 0.18 m	Terminal positioning precision
Mean time to goal	16.8 s	Navigation efficiency
Training mean reward	22.2 ± 0.5	At iteration 1 500
Training fall rate	5.95%	At iteration 1 500

Figure 2 shows the evolution of key training metrics over 1 500 iterations.



(a) Episode reward

(b) Episode length

(c) Distance to goal

(d) Actor/critic losses

Fig. 2: Base locomotion training curves over 1 500 PPO iterations. Reward and episode length converge monotonically; distance to goal decreases steadily; actor and critic losses stabilize, confirming policy convergence.

The learned behavior exhibits several desirable characteristics:

- Stable alternating gait patterns
- Consistent forward motion toward target goals
- Robust recovery from small perturbations

The policy operates directly at the joint command level across 29 actuated degrees of freedom, demonstrating that end-to-end reinforcement learning can synthesize coordinated locomotion behaviors without explicit trajectory planning or predefined gait models. Figure 3 shows the goal sampling distribution and representative trajectories observed during evaluation.

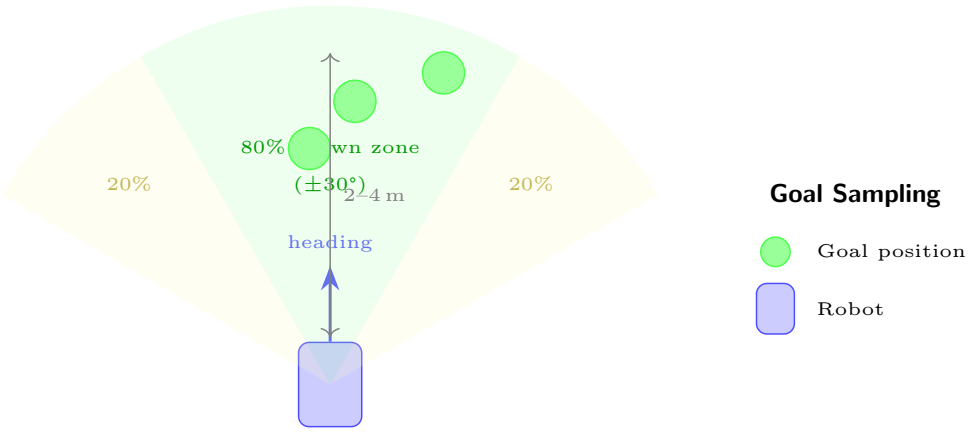


Fig. 3: Goal sampling distribution and representative evaluation trajectories. Left: forward-biased initial goal sampling (65% front hemisphere). Right: example trajectories showing the robot navigating to sequentially sampled goals while maintaining upright posture.

The robot consistently moves toward the goal location while maintaining upright posture.

These results provide empirical support for the hypothesis that goal-directed locomotion for humanoid robots can be achieved using a single learned policy operating on proprioceptive observations, without requiring hierarchical control decomposition.

5.2 Emergent Locomotion Behaviors

During training, the locomotion strategy evolved gradually through reinforcement learning optimization.

Early training phases were characterized by unstable behaviors including:

- Oscillatory joint motions
- Frequent loss of balance
- Inefficient movement patterns

As training progressed, the agent discovered locomotion strategies that stabilized the robot's center of mass and coordinated lower-limb motion.

The resulting locomotion pattern resembles a simplified walking gait characterized by:

- alternating leg motion,
- forward center-of-mass displacement,
- periodic ground contact.

Although the learned gait is not explicitly constrained by human-like biomechanics, it achieves stable dynamic locomotion within the simulated environment.

These findings are consistent with previous work demonstrating that deep reinforcement learning can discover effective locomotion strategies in high-dimensional robotic systems.

5.3 Simulation-to-Simulation Validation Using ROS 2 Middleware

A central contribution of this work is the introduction of a communication-realistic simulation-to-simulation validation stage.

In this configuration, the trained locomotion policy was integrated into a control pipeline based on ROS 2 Humble, replicating the communication architecture used by the physical robot.

Unlike traditional reinforcement learning evaluation—where the policy interacts directly with the simulator—this setup introduces:

- message-based control loops
- asynchronous communication
- middleware-level data exchange

The policy was deployed as a ROS 2 node responsible for generating joint commands based on sensor observations.

The digital twin implemented in NVIDIA Isaac Lab provided simulated sensor data through ROS 2 topics, allowing the entire control pipeline to operate under conditions similar to those expected in real robotic deployments.

The results demonstrate that the learned locomotion policy remains functional under this middleware-based execution architecture. This finding is important because communication delays, message serialization, and asynchronous control loops often introduce discrepancies between training environments and real robotic systems.

By validating the policy in a communication-realistic environment prior to hardware deployment, the proposed framework reduces risks associated with the simulation-to-reality transition.

5.4 Exploratory Results for Social Navigation

An exploratory experiment was conducted to evaluate whether the reinforcement learning framework could be extended to support obstacle-aware navigation.

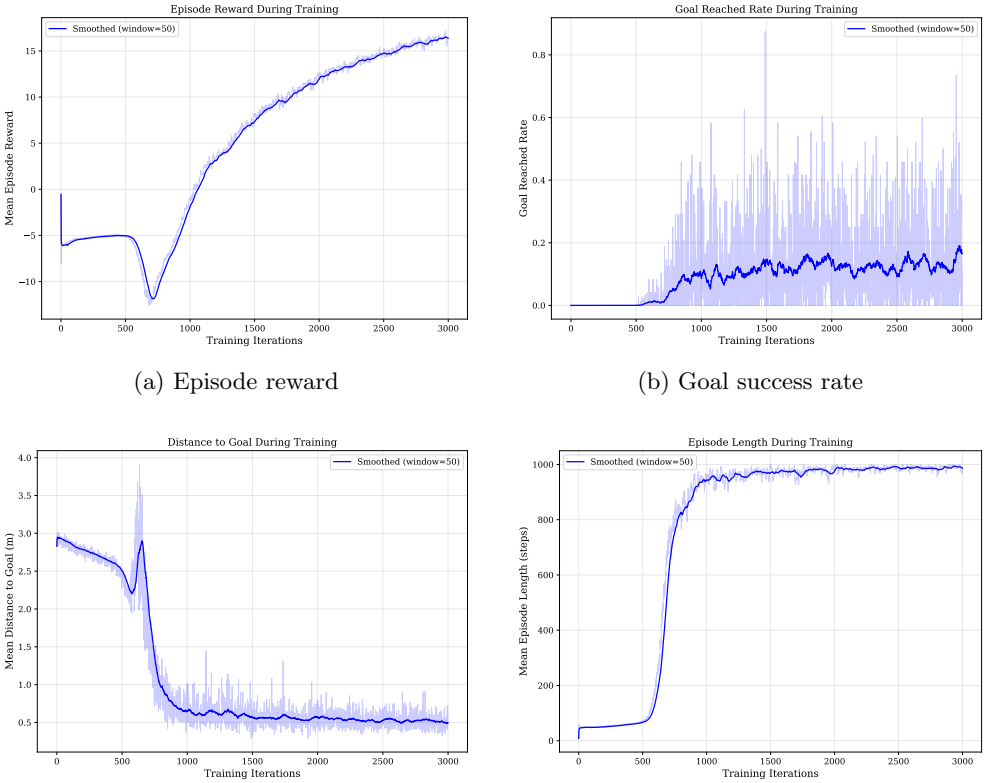
In this configuration, the observation space was augmented with LiDAR perception data and additional reward terms were introduced to penalize collisions and encourage obstacle avoidance.

However, the resulting policy achieved only 1

Quantitative analysis identifies the root cause: at a representative obstacle encounter (robot 2 m from goal, obstacle blocking the path), the goal-seeking reward gradient has magnitude ≈ 0.07 while the warning-zone proximity gradient has magnitude ≈ 0.18. The obstacle-avoidance gradient exceeds the goal-seeking gradient by a factor of $2.6\times$, causing the policy to retreat rather than circumnavigate. This imbalance is a direct consequence of the penalty weights (-0.5 at 1.5 m warning zone, -2.0 at 0.8 m personal space) relative to the combined goal-distance reward weight ($+0.5$

total), and suggests that future formulations should either increase goal-reward magnitude, introduce curriculum learning (obstacle-free warm-up followed by progressive obstacle insertion), or adopt a hierarchical approach that decouples goal navigation from collision avoidance.

Figures **??** and **??** show the training metrics for the social navigation experiment, confirming the failure to converge.



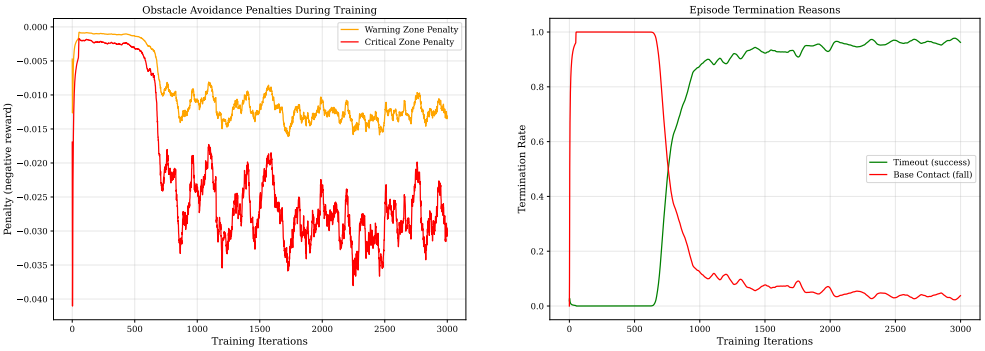
(a) Episode reward

(b) Goal success rate

(c) Distance to goal

(d) Episode length

Fig. 4: Social navigation performance metrics. Goal success rate (b) never exceeds 1% and distance to goal (c) fails to decrease, indicating that the policy does not converge toward effective navigation.



(a) Obstacle proximity penalties

(b) Termination causes

Fig. 5: Social navigation penalty and termination analysis. Obstacle proximity penalties (a) dominate the reward signal throughout training, confirming the 2.6× gradient imbalance. Frequent early terminations (b) reflect the policy retreating from obstacles rather than navigating around them.

5.5 Implications for Humanoid Robotics Research

The results obtained in this work highlight several important implications for humanoid robotics research.

First, the successful locomotion policy demonstrates that end-to-end reinforcement learning can generate stable goal-directed locomotion behaviors for high-degree-of-freedom humanoid robots, even when operating directly at the joint command level.

Second, the proposed simulation-to-simulation validation architecture provides a practical intermediate step between reinforcement learning training and physical deployment. This approach enables researchers to verify the integration of learning-based controllers within realistic robotic communication architectures before executing experiments on expensive hardware platforms.

Finally, the challenges encountered during the social navigation experiments illustrate the complexity of multi-objective learning problems in humanoid robotics. Reward design remains a critical research challenge when attempting to combine locomotion control with perception-driven navigation behaviors.

Together, these findings support the value of integrated development pipelines that combine digital twins, reinforcement learning, and middleware-based validation for the safe and scalable development of humanoid robot capabilities.

6 Implications for Intelligent and Robotic Systems

The findings of this work provide several insights relevant to the broader field of intelligent robotic systems, particularly in the context of learning-based control for complex embodied agents.

6.1 End-to-End Learning for High-DOF Humanoid Control

The successful locomotion policy obtained in this study supports the feasibility of end-to-end reinforcement learning for high-degree-of-freedom humanoid robots.

Traditional humanoid control architectures typically rely on hierarchical decomposition, separating the control stack into multiple layers such as:

- global path planning,
- trajectory generation,
- gait synthesis,
- low-level joint control.

While this structure offers interpretability and modularity, it also introduces engineering complexity and limits the ability of the system to adapt autonomously to new dynamics or environments.

The results presented in this work demonstrate that a single policy trained using Proximal Policy Optimization can learn coordinated locomotion behaviors operating directly on joint-level commands across 29 degrees of freedom. This suggests that, under appropriate training conditions, reinforcement learning can implicitly discover locomotion strategies that traditionally require carefully designed model-based controllers.

Although hierarchical approaches remain valuable in many contexts, the results indicate that end-to-end policies represent a viable alternative for certain classes of humanoid locomotion problems, particularly when supported by high-fidelity simulation environments.

6.2 The Role of Digital Twins in Robotics Development

Another important implication of this work concerns the role of digital twin environments in the development of intelligent robotic systems.

The integration of a high-fidelity digital twin implemented in NVIDIA Isaac Lab enabled large-scale training and evaluation of locomotion policies without exposing the physical robot to potential damage during early experimentation phases.

This capability is particularly significant for humanoid platforms such as the Unitree G1 humanoid robot, where falls or unstable behaviors can result in costly repairs and operational downtime.

The results suggest that digital twins can serve not only as simulation tools for algorithm development but also as central components of the robotics experimentation lifecycle, enabling:

- large-scale reinforcement learning training,
- safe evaluation of control policies,
- reproducible experimentation across research groups.

Such infrastructures are especially valuable in educational and research institutions where access to physical humanoid robots is limited.

6.3 Communication-Realistic Validation for Learning-Based Controllers

A key contribution of this work is the introduction of a communication-realistic simulation-to-simulation validation stage.

In many reinforcement learning studies, trained policies are evaluated within the same simulator used during training. While this approach provides valuable performance insights, it often ignores an important aspect of real robotic systems: the communication middleware that connects perception, control, and actuation modules.

By integrating the learned locomotion policy into a middleware-based execution pipeline using ROS 2, the proposed framework introduces an intermediate validation stage where policies are evaluated under realistic communication constraints.

This step exposes the control architecture to factors that commonly affect real robotic deployments, including:

- asynchronous message passing,
- serialization overhead,
- distributed control loops.

The results suggest that such sim-to-sim validation pipelines can reduce integration risks prior to hardware deployment, serving as a practical bridge between reinforcement learning training and physical experimentation.

6.4 Lessons for Multi-Objective Learning in Humanoid Navigation

The exploratory experiments on obstacle-aware navigation highlight the challenges associated with multi-objective reinforcement learning in humanoid systems.

The introduction of LiDAR perception and obstacle avoidance rewards resulted in a drastic reduction in navigation performance, despite the policy maintaining stable locomotion behaviors.

This outcome illustrates an important methodological lesson: combining multiple objectives—such as goal reaching, collision avoidance, and locomotion stability—within a single reward function can produce conflicting optimization gradients that hinder learning convergence.

The diagnostic analysis performed in this work suggests that effective humanoid navigation policies may require more structured training strategies, including:

- curriculum learning,
- hierarchical reinforcement learning,
- decoupled reward mechanisms for navigation and locomotion.

Understanding how to balance these objectives remains an open research challenge for the development of autonomous humanoid systems operating in human environments.

6.5 Implications for Robotics Education and Research Infrastructure

Beyond the technical contributions, the proposed framework also has implications for robotics education and research infrastructure.

Many universities face similar constraints to those described in this study: limited access to expensive robotic hardware combined with a growing demand for hands-on learning in robotics and artificial intelligence.

By combining reinforcement learning training, digital twin environments, and middleware-based validation, the proposed framework provides a scalable development pipeline that allows large student communities to experiment with humanoid robotics without requiring continuous access to physical robots.

This approach can significantly expand the accessibility of advanced robotics experimentation while reducing risks associated with early-stage algorithm development.

7 Conclusion

This paper presented an integrated framework for training and validating humanoid locomotion policies using digital twins and deep reinforcement learning, addressing the challenge of safe policy development in resource-constrained research environments.

End-to-end navigation without hierarchical decomposition. The base locomotion policy achieved an 87% goal success rate with stable bipedal gait, demonstrating that a direct joint-level policy trained via Proximal Policy Optimization can produce effective goal-directed navigation across 29 degrees of freedom without separate trajectory planning, gait generation, or motion-primitive modules. Key design elements enabling this result include forward-biased goal sampling, heading-aware velocity scaling, and dual-scale exponential rewards that provide gradient across both long-range approach and precision terminal positioning. The policy maintains a 5.95% fall rate while navigating to goals with 0.42 ± 0.18 m average precision.

Social navigation: a documented failure. The exploratory obstacle-aware navigation task achieved only 1% goal success rate despite functional LiDAR integration and dynamic obstacle detection. Analysis identifies the root cause as a gradient imbalance: proximity penalties create reward gradients 2.6× stronger than goal-seeking rewards (gradient magnitudes 0.18 vs. 0.07), causing the policy to retreat from obstacles rather than navigate around them. This documented failure provides diagnostic value that informs future reward formulations for multi-objective humanoid navigation.

Sim-to-sim infrastructure verified. The simulation-to-simulation validation framework using ROS 2 Humble is functionally verified: all topics publish at expected rates with correct message formats, enabling future latency characterization and staged validation prior to physical deployment.

The primary contributions of this work are: (1) an end-to-end 29-DOF goal-directed locomotion policy achieving 87% goal success without hierarchical decomposition; (2) a sim-to-sim validation stage using authentic ROS 2 middleware protocols; (3) a containerized deployment pipeline enabling reproducible experimentation in resource-constrained environments; and (4) documented analysis of multi-objective social

navigation failure that identifies specific gradient imbalances to guide future reward design.

Key limitations include simulation-only validation (no physical transfer), single training runs per configuration, no ablation study on reward components, and uncharacterized ROS 2 bridge latency. Future priorities are physical sim-to-real deployment, revised social navigation rewards with curriculum or hierarchical learning, and quantitative latency characterization of the ROS 2 bridge.

Supplementary information. Not applicable.

Acknowledgements. The authors thank the Universidad de los Andes MISIS program for computational resources and research support.

Declarations

Funding. This work was supported by Universidad de los Andes (MISIS Program) through research scholarship funding.

Competing interests. The authors declare no competing interests.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. The source code, training configurations, and experiment logs are available at the project repository. Trained model checkpoints are available upon reasonable request.

Materials availability. Not applicable.

Code availability. The source code implementing the digital twin environment, reinforcement learning training pipeline, and ROS 2 sim-to-sim validation framework is available at the project repository. Access details are provided upon reasonable request.

Authors' contributions. Juan Andrés Méndez: conceptualization, methodology, software, investigation, formal analysis, writing — original draft. Carlos Lozano-Garzon: supervision, writing — review and editing.

References

[1] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, C.D., Tsounis, V., Koltun, V., Hutter, M.: Learning agile and dynamic motor skills for legged robots. Science Robotics **4**(26), 5872 (2019) https://doi.org/10.1126/scirobotics.aau5872

[2] Radosavovic, I., Xiao, T., Zhang, B., Darrell, T., Malik, J., Sreenath, K.: Real-world humanoid locomotion with reinforcement learning. Science Robotics **9**(89), 9579 (2024) https://doi.org/10.1126/scirobotics.adi9579

[3] Mittal, M., Yu, C., Kumar, Q., Kim, J., Rudin, N., Hoeller, D., Lee, J.L., Farshidian, F., Hutter, M.: Orbit: A unified simulation framework for interactive robot learning environments. IEEE Robotics and Automation Letters **8**(6), 3740–3747 (2023) https://doi.org/10.1109/LRA.2023.3306588

[4] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347. https://arxiv.org/abs/1707.06347 (2017)

[5] Rudin, N., Hoeller, D., Reist, P., Hutter, M.: Learning to walk in minutes using massively parallel deep reinforcement learning. In: Proceedings of the 5th Conference on Robot Learning (CoRL 2022), vol. 164, pp. 91–100 (2022). arXiv:2109.11978

[6] Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., Hutter, M.: Learning quadrupedal locomotion over challenging terrain. Science Robotics **5**(47), 5986 (2020) https://doi.org/10.1126/scirobotics.abc5986

[7] Gu, X., Wang, Y.-J., Chen, J.: Humanoid-Gym: Reinforcement Learning for Humanoid Robot with Zero-Shot Sim2Real Transfer. arXiv preprint arXiv:2404.05695. https://arxiv.org/abs/2404.05695 (2024)

[8] Cheng, X., Ji, Y., Chen, J., Yang, R., Yang, G., Wang, X.: Expressive Whole-Body Control for Humanoid Robots. arXiv preprint arXiv:2402.16796. https://arxiv.org/abs/2402.16796 (2024)

[9] Peng, X.B., Abbeel, P., Levine, S., Panne, M.: DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. ACM Transactions on Graphics **37**(4), 143–114314 (2018) https://doi.org/10.1145/3197517.3201311

[10] Grieves, M.W.: Digital twin: Manufacturing excellence through virtual factory replication. White paper, Michael W. Grieves LLC (2014). https://theengineer.markallengroup.com/production/content/uploads/2014/12/Digital_Twin_White_Paper_Dr_Grieves.pdf

[11] Tao, F., Zhang, H., Liu, A., Nee, A.Y.C.: Digital twin in industry: State-of-the-Art. IEEE Transactions on Industrial Informatics **15**(4), 2405–2415 (2019) https://doi.org/10.1109/TII.2018.2873186

[12] NVIDIA: Isaac Sim: Robotics Simulation and Synthetic Data Generation. NVIDIA Corporation. https://developer.nvidia.com/isaac/sim (2024)

[13] Sueldo, C., Perez Colo, I., De Paula, M., Villar, S., Acosta, G.: ROS-based architecture for fast digital twin development of smart manufacturing robotized systems. Annals of Operations Research **322**, 153–178 (2023) https://doi.org/10.1007/s10479-022-04759-4

[14] Zhao, W., Queralta, J.P., Westerlund, T.: Sim-to-real transfer in deep reinforcement learning for robotics: A survey. In: 2020 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 737–744 (2020). https://doi.org/10.1109/SSCI47803.2020.9308468

[15] Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P.: Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 23–30 (2017). https://doi.org/10.1109/IROS.2017.8202133 . arXiv:1703.06907

[16] Kumar, A., Fu, Z., Pathak, D., Malik, J.: RMA: Rapid motor adaptation for legged robots. In: Robotics: Science and Systems XVII (2021). https://doi.org/10.15607/RSS.2021.XVII.011 . arXiv:2107.04034

[17] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., Woodall, W.: Robot operating system 2: Design, architecture, and uses in the wild. Science Robotics **7**(66), 6074 (2022) https://doi.org/10.1126/scirobotics.abm6074

[18] Kober, J., Bagnell, J.A., Peters, J.: Reinforcement learning in robotics: A survey. International Journal of Robotics Research **32**(11), 1238–1274 (2013) https://doi.org/10.1177/0278364913495721